

Playfair Cipher

Aim

To implement the Playfair cipher, a digraph substitution cipher that encrypts letters two at a time using a 5x5 key matrix. The alphabet is fit into 25 cells by merging J into I; text is uppercased and non-letters are dropped. Repeated letters in a pair are split with an X, and an odd-length message is padded with a trailing X.

Algorithm

```
1 Function Encrypt(plain_text, key):
2   grid ← 5x5 matrix from key then A..Z (J merged into I)
3   text ← sanitize(plain_text)                                // uppercase, letters only, J to I
4   digraphs ← split text into pairs                          // insert X between equal letters; pad tail with X
5   cipher ← ""
6   foreach pair (a, b) in digraphs do
7     (r1, c1) ← position of a
8     (r2, c2) ← position of b
9     if r1 = r2 then
10      c1 ← (c1 + 1) mod 5
11      c2 ← (c2 + 1) mod 5                                // same row:  shift right
12    else if c1 = c2 then
13      r1 ← (r1 + 1) mod 5
14      r2 ← (r2 + 1) mod 5                                // same col:  shift down
15    else
16      swap(c1, c2)                                         // rectangle:  swap columns
17    append grid[r1][c1] and grid[r2][c2] to cipher
18  return cipher
```

```
1 Function Decrypt(cipher_text, key):
2   grid, digraphs built as in Encrypt
3   plain ← ""
4   foreach pair (a, b) in digraphs do
5     (r1, c1) ← position of a
6     (r2, c2) ← position of b
7     if r1 = r2 then
8      c1 ← (c1 - 1) mod 5
9      c2 ← (c2 - 1) mod 5                                // same row:  shift left
10    else if c1 = c2 then
11      r1 ← (r1 - 1) mod 5
12      r2 ← (r2 - 1) mod 5                                // same col:  shift up
13    else
14      swap(c1, c2)                                         // rectangle:  swap columns
15    append grid[r1][c1] and grid[r2][c2] to plain
16  return plain
```

Output

Encrypt — key MONARCHY, plaintext BALLOON

The key builds this 5x5 matrix (row, col are 0-based):

	0	1	2	3	4
0	M	O	N	A	R
1	C	H	Y	B	D
2	E	F	G	I	K
3	L	P	Q	S	T
4	U	V	W	X	Z

BALLOON splits into digraphs BA LX LO ON (the double LL is split by an X). Each pair is enciphered by its rule:

Digraph	Positions	Rule	New positions	Cipher
B A	(1,3) (0,3)	same col -> shift down	(2,3) (1,3)	I B
L X	(3,0) (4,3)	rectangle -> swap cols	(3,3) (4,0)	S U
L O	(3,0) (0,1)	rectangle -> swap cols	(3,1) (0,0)	P M
O N	(0,1) (0,2)	same row -> shift right	(0,2) (0,3)	N A

Result: BALLOON -> IBSUPMNA

Decrypt — key MONARCHY, ciphertext IBSUPMNA

The same matrix and pairing are used; row/column rules move the opposite way.

Digraph	Positions	Rule	New positions	Plain
I B	(2,3) (1,3)	same col -> shift up	(1,3) (0,3)	B A
S U	(3,3) (4,0)	rectangle -> swap cols	(3,0) (4,3)	L X
P M	(3,1) (0,0)	rectangle -> swap cols	(3,0) (0,1)	L O
N A	(0,2) (0,3)	same row -> shift left	(0,1) (0,2)	O N

Result: IBSUPMNA -> BALXLOON

The recovered BALXLOON still holds the padding X inserted during encryption; Playfair cannot tell filler Xs from real ones, so removing them is left to the reader.